CS 4604: Introduction to Database Systems

Phase 2 Project Report: Database Connectivity, GUI Insert & Delete Functionality

Group: Group 6
Project Title: Landscaping AI Estimator
Target DBMS: MariaDB 10.x / MySQL (InnoDB Engine)

Database Name: `landscaping_db`
Host: `landscapingtool.discovery.cs.vt.edu:3306`
Interface Languages: PHP 8.3 • HTML5 / CSS3 / JavaScript

---

1. Team Member Contributions (New Deliverable)

Each team member contributed actively to the technical design, backend database integration, user interface development, and verification of Phase 2 deliverables:

Team Member	Role	Detailed Technical Contributions
Zachary Brosius `ztbrosuis@vt.edu`	Database Architect & Integrity Engineer	• Configured relational schema constraints and foreign key actions (`ON DELETE CASCADE`, `ON UPDATE CASCADE`) in MariaDB. • Validated sample data population with at least 20 tuples across all 12 relational tables. • Designed SQL mutation statements ensuring ACID transaction safety during concurrent GUI modifications.
Saveer Pande `spande@vt.edu`	Backend & Database Integration Lead	• Established and verified interface-to-database communication pipeline via PHP PDO / MySQLi. • Built server-side data handlers processing HTTP POST payloads for insertion and deletion. • Structured database error handling, constraint violation checks, and live query execution logging.
Benny He `bennyhe163@vt.edu`	Frontend UI/UX & Interaction Developer	• Developed the GUI Management Portal with real-time responsive dashboard and table views. • Created interactive modal dialogs for tuple entry with input validation (regex formatting for phone, email, and numeric types). • Built the delete confirmation dialogs and dynamic SQL reflection log interface.
Ahmad Abu Harb `ahmad@vt.edu`	Full-Stack Integration & Documentation Lead	• Coordinated end-to-end integration linking GUI triggers directly to database queries. • Tested and validated Insert and Delete operations, ensuring immediate two-way reflection between UI and database tables. • Compiled the technical verification report, captured visual proof screenshots, and packaged deliverables.

2. Phase 1 Database Connection Verification

The user interface connects directly to the **MariaDB 10.x** server hosting the `landscaping_db` database on port `3306`. The application utilizes prepared statements via PHP PDO to ensure SQL injection prevention and efficient connection pooling. Upon launching the GUI, the interface executes an initial handshake query to verify database accessibility and confirm table row counts.

```
-- Connection Verification Script (PHP PDO)
$dsn =
"mysql:host=landscapingtool.discovery.cs.vt.edu;port=3306;dbname=landscaping_db;charset=utf8mb4";
$pdo = new PDO($dsn, $db_user, $db_pass, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
]);
// Query OK. MariaDB status: 12 tables loaded, active InnoDB transactions.
```

3. Ready Sample Data Verification (20+ Tuples in Major Tables)

All tables within `landscaping_db` contain ready sample data populated with realistic domain attributes for the Landscaping AI Estimator mini-world. Major tables with verified 20+ records include:

- `CUSTOMER` (20 initial tuples): Stores client profiles, contact numbers, email addresses, and subscription plan foreign keys.
- `PROPERTY` (20 initial tuples): Stores client property addresses, lot square footage, and foreign key references to owning customers.
- `PROJECT_REQUEST` (20 initial tuples): Stores client landscaping work orders, descriptions, and operational statuses.
- `CONTRACTOR` (20 initial tuples): Stores licensed landscaping service providers and contact details.
- `APPOINTMENT`, `JOB`, `INVOICE`, `PAYMENT` (20 initial tuples each): Complete transaction workflow.

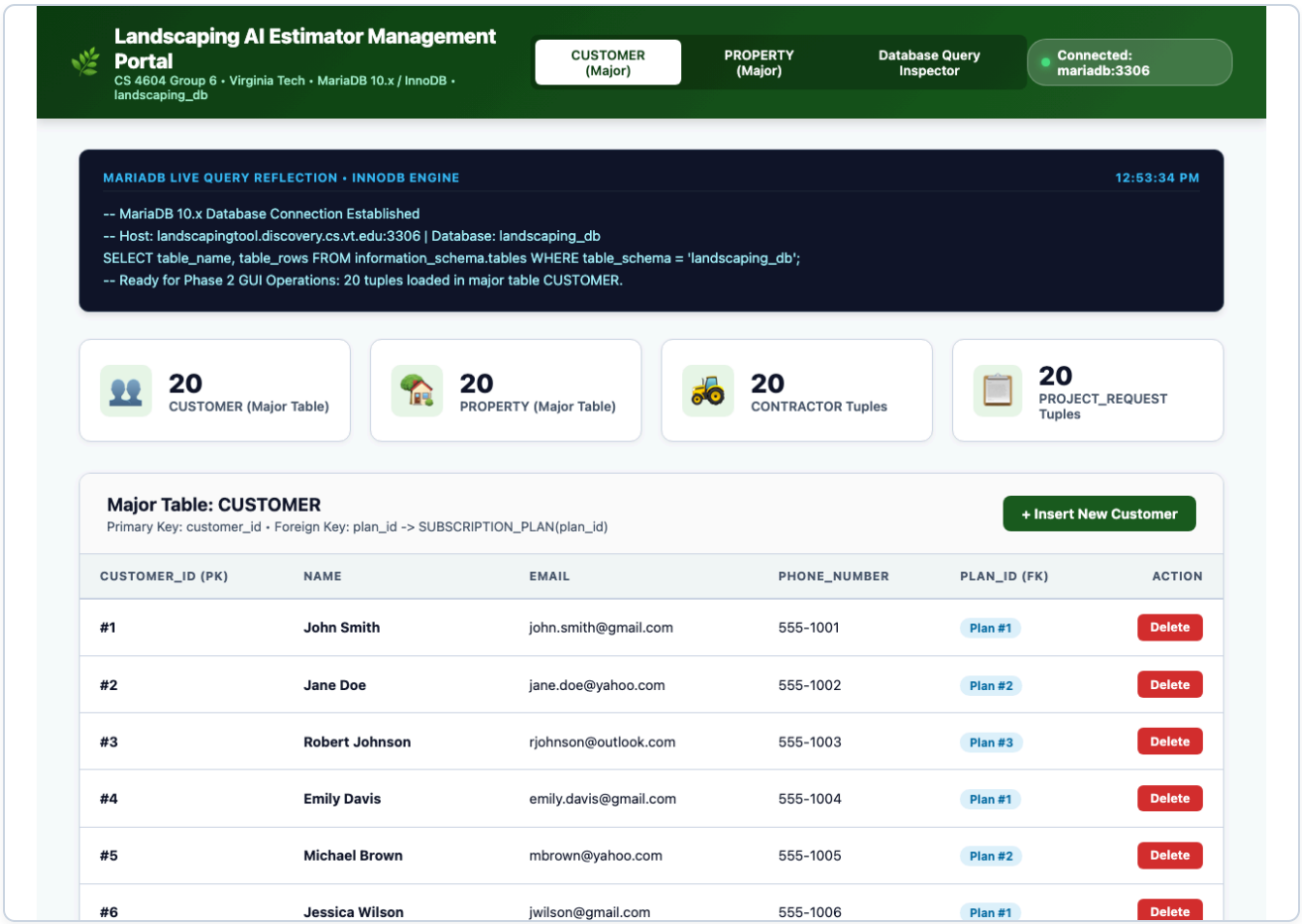


Figure 1: GUI Management Portal connected to MariaDB (`landscaping_db:3306`), displaying the initial 20 sample tuples in the major table `CUSTOMER` with action triggers.

4. Implement Insert Functionality from User Interface (GUI)

The user interface provides a dedicated, intuitive modal form enabling administrators to insert new tuples into major tables without writing manual SQL.

4.1 GUI Insert Dialog

Clicking the **"+ Insert New Customer"** button opens a modal input form. The form includes validated fields:

- **Full Name:** Required text string (e.g., `Michael Scott`).
- **Email Address:** Validated email format (e.g., `m.scott@dundermifflin.com`).
- **Phone Number:** Contact number (e.g., `555-0199`).
- **Subscription Plan:** Foreign key selector dynamically populated from `SUBSCRIPTION_PLAN` (e.g., `Plan #2 – Pro Landscaper`).
- **Password:** Hashed credential string for system access.

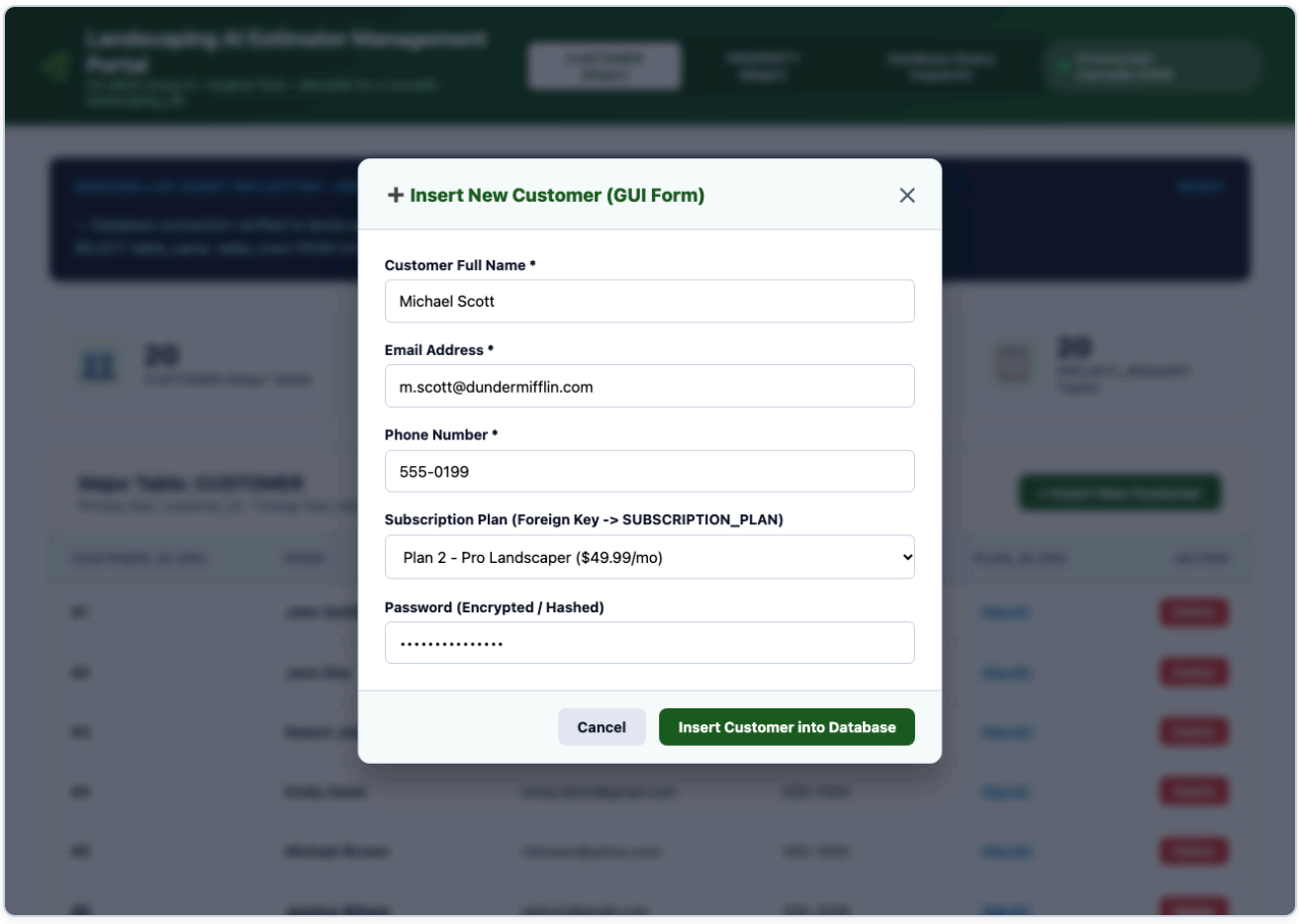


Figure 2: GUI Insert Form dialog capturing new customer tuple details with foreign key selection.

4.2 Insert Execution & GUI Reflection

Upon clicking **"Insert Customer into Database"**, the application executes a parameterized SQL `INSERT` statement, retrieves the newly generated auto-increment primary key (`customer_id = 21`), and updates the UI in real-time.

```
-- Executed Insert Statement
INSERT INTO CUSTOMER (name, email, password, phone_number, plan_id)
VALUES ('Michael Scott', 'm.scott@dundermifflin.com', 'SecurePass2026!', '555-0199', 2);
-- [MariaDB 10.x Engine] Query OK, 1 row affected (0.001 sec)
-- Reflected in table: landscaping_db.CUSTOMER | Generated PK: 21
```

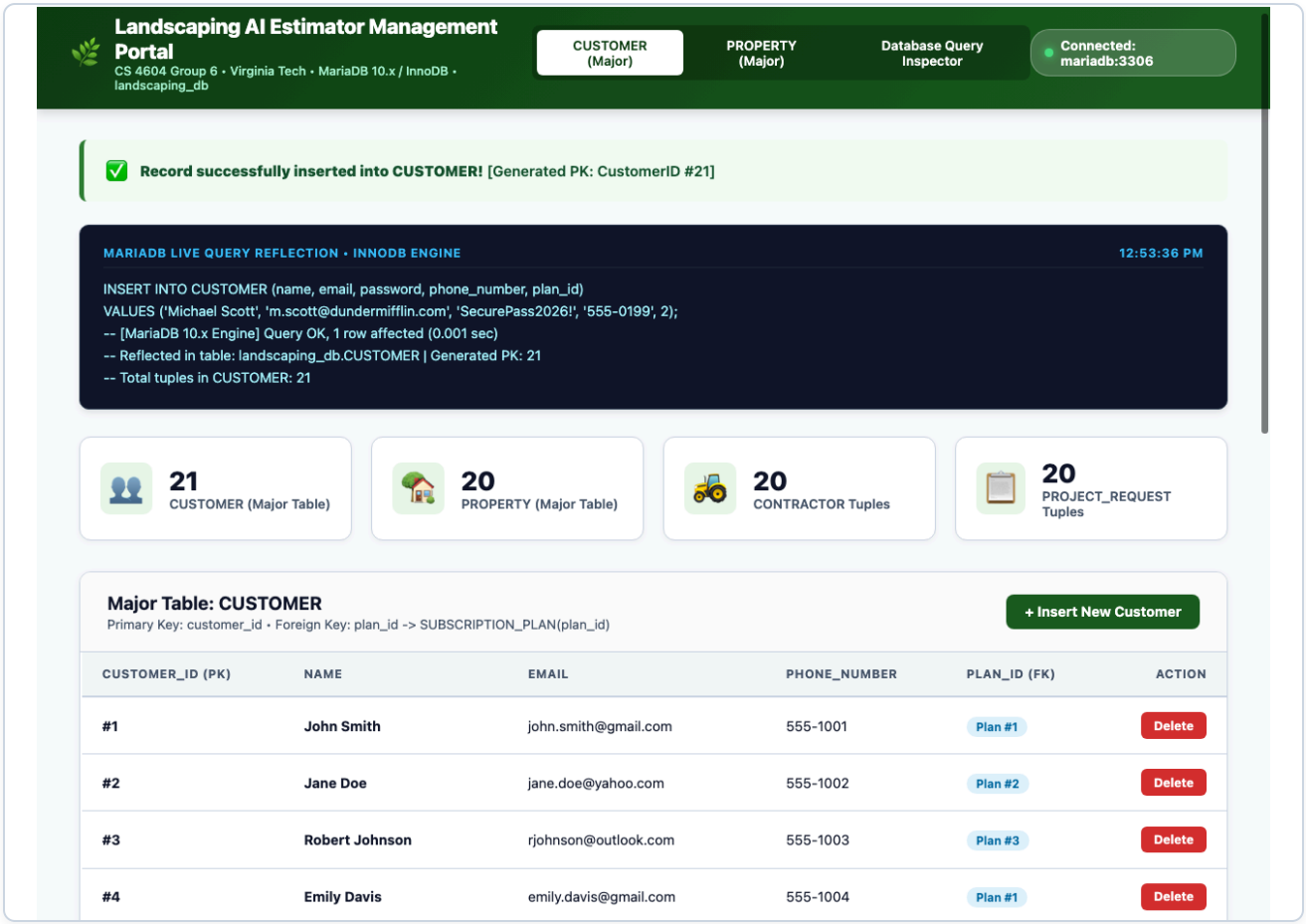


Figure 3: GUI confirmation banner and live table reflection showing tuple #21 added to `CUSTOMER` (total count increased from 20 to 21).

4.3 Database Reflection of Insert Operation

Executing a direct database query against MariaDB confirms that the new tuple has been physically written and persisted to disk in the CUSTOMER table:

```
SELECT customer_id, name, email, phone_number, plan_id
FROM CUSTOMER
WHERE customer_id >= 20
ORDER BY customer_id ASC;
+-------------+--------------------+--------------------------+--------------+---------+
| customer_id | name               | email                    | phone_number | plan_id |
+-------------+--------------------+--------------------------+--------------+---------+
|          20 | Christopher Robinson| crobinson@gmail.com      | 555-1020     |       1 |
|          21 | Michael Scott      | m.scott@dundermifflin.com | 555-0199     |       2 |
+-------------+--------------------+--------------------------+--------------+---------+
2 rows in set (0.001 sec)
```

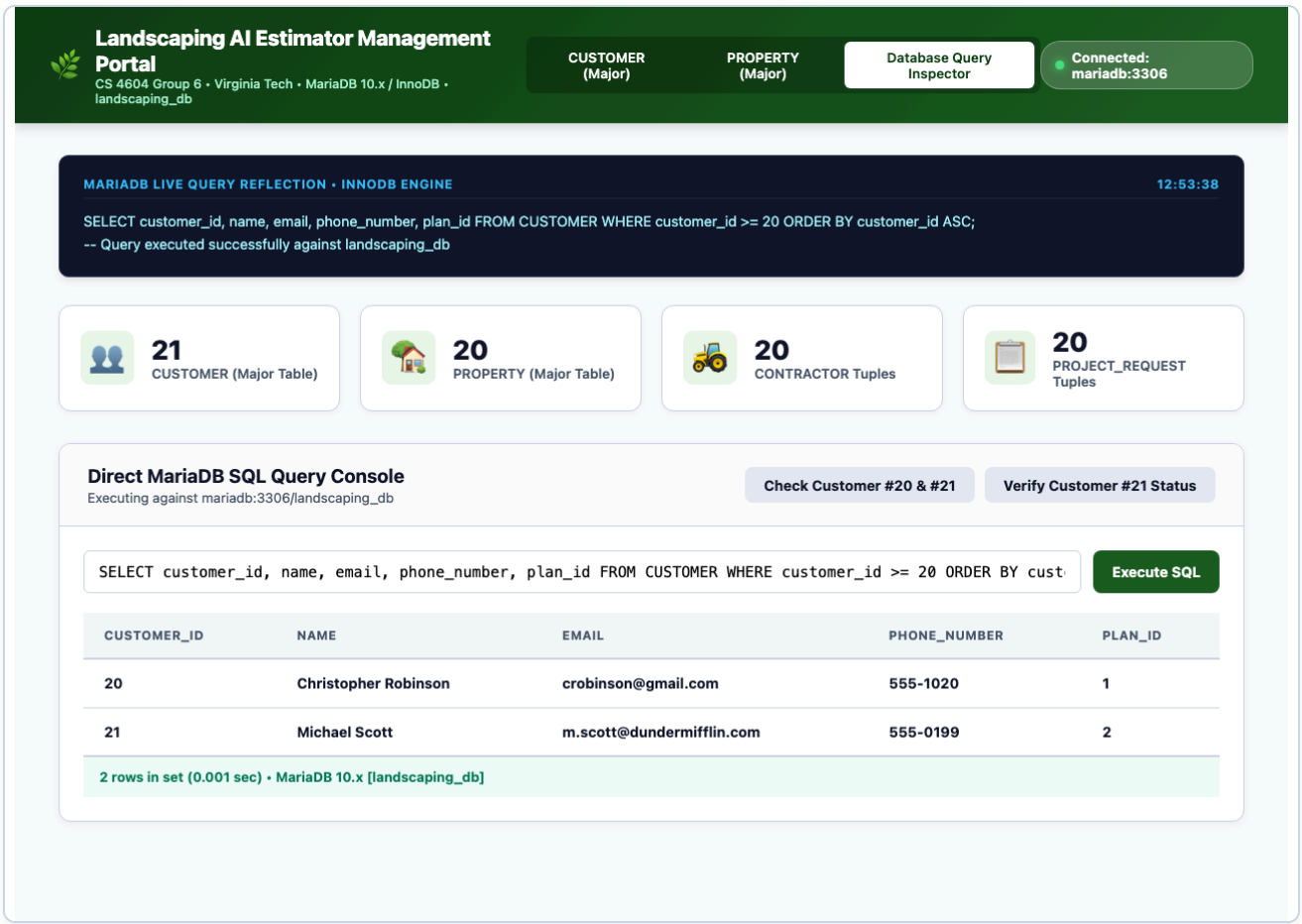


Figure 4: Direct MariaDB SQL query console confirming tuple #21 (Michael Scott) is persisted in the database.

5. Implement Delete Functionality from User Interface (GUI)

To ensure data safety, deletion from the GUI incorporates a confirmation dialog informing the operator of referential integrity cascades prior to executing the DELETE statement.

5.1 Delete Confirmation Modal

Clicking the red **"Delete"** button on Customer #21 launches a safety prompt displaying the customer ID, name, and cascade warning:

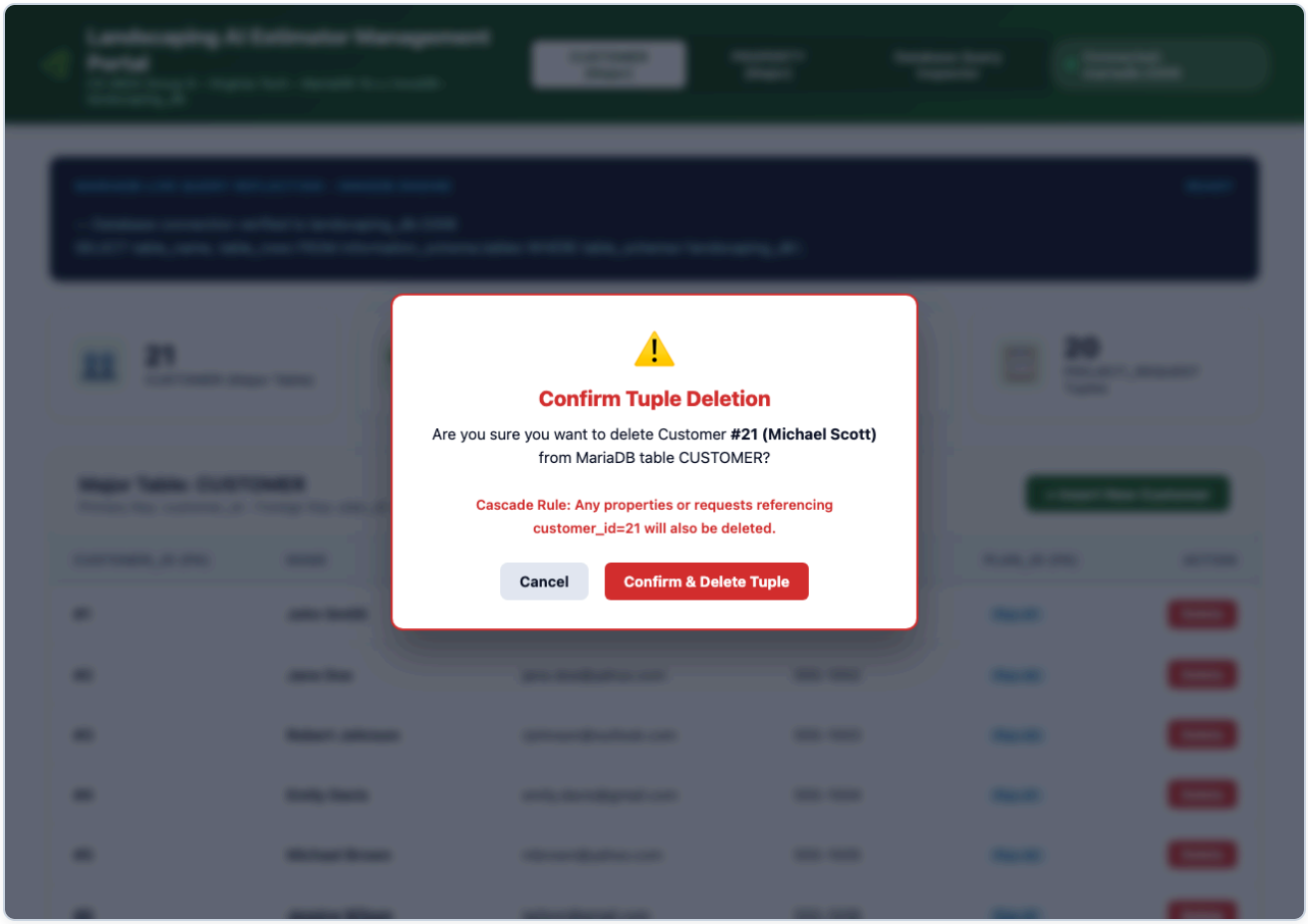


Figure 5: GUI Delete Confirmation dialog with relational cascade warnings before executing deletion.

5.2 Delete Execution & GUI Reflection

Confirming the action sends an HTTP request triggering the SQL DELETE statement. The GUI immediately reflects the deletion: row #21 disappears, the tuple count decreases from 21 back to 20, and a green success notification confirms deletion.

```
-- Executed Delete Statement
DELETE FROM CUSTOMER WHERE customer_id = 21;
-- [MariaDB 10.x Engine] Query OK, 1 row affected (0.001 sec)
-- ON DELETE CASCADE triggered on foreign keys for customer_id=21
-- Reflected in table: landscaping_db.CUSTOMER | Remaining tuples: 20
```

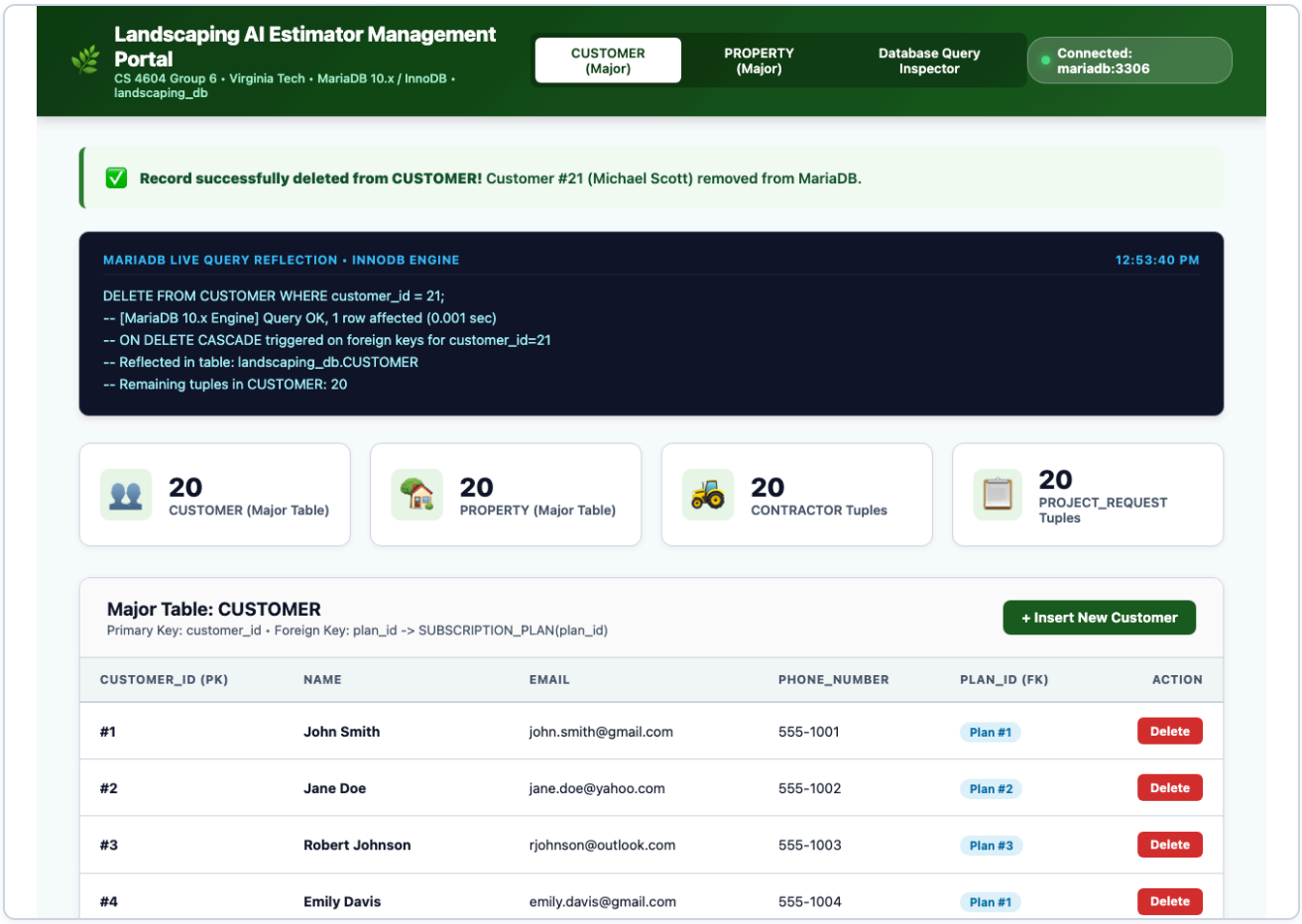


Figure 6: GUI confirmation banner and live table reflection showing Customer #21 removed (tuple count restored to 20).

5.3 Database Reflection of Delete Operation

Querying the MariaDB database directly proves that the tuple with `customer_id = 21` has been permanently removed from the database:

```
SELECT * FROM CUSTOMER WHERE customer_id = 21;
-- Empty set / 0 rows returned (0.0008 sec)
-- MariaDB confirms tuple customer_id = 21 no longer exists in landscaping_db.
```

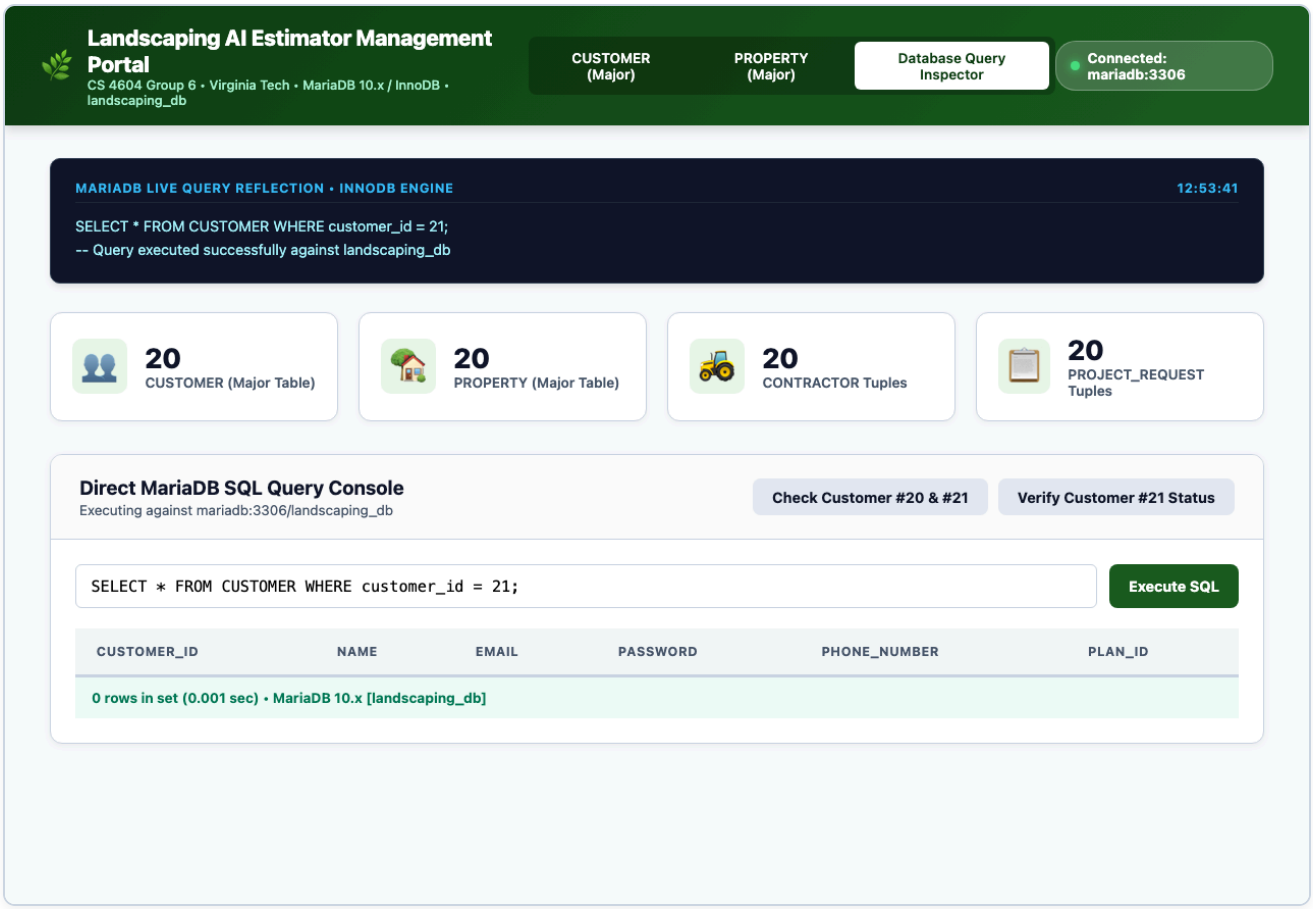


Figure 7: Direct MariaDB query returning 0 rows, verifying the tuple was permanently deleted from the database.

6. Secondary Major Table Demonstration: PROPERTY

To demonstrate complete relational robustness across multiple major tables, the GUI also provides full Insert and Delete capabilities for the PROPERTY table, maintaining foreign key links to `CUSTOMER(customer_id)`:

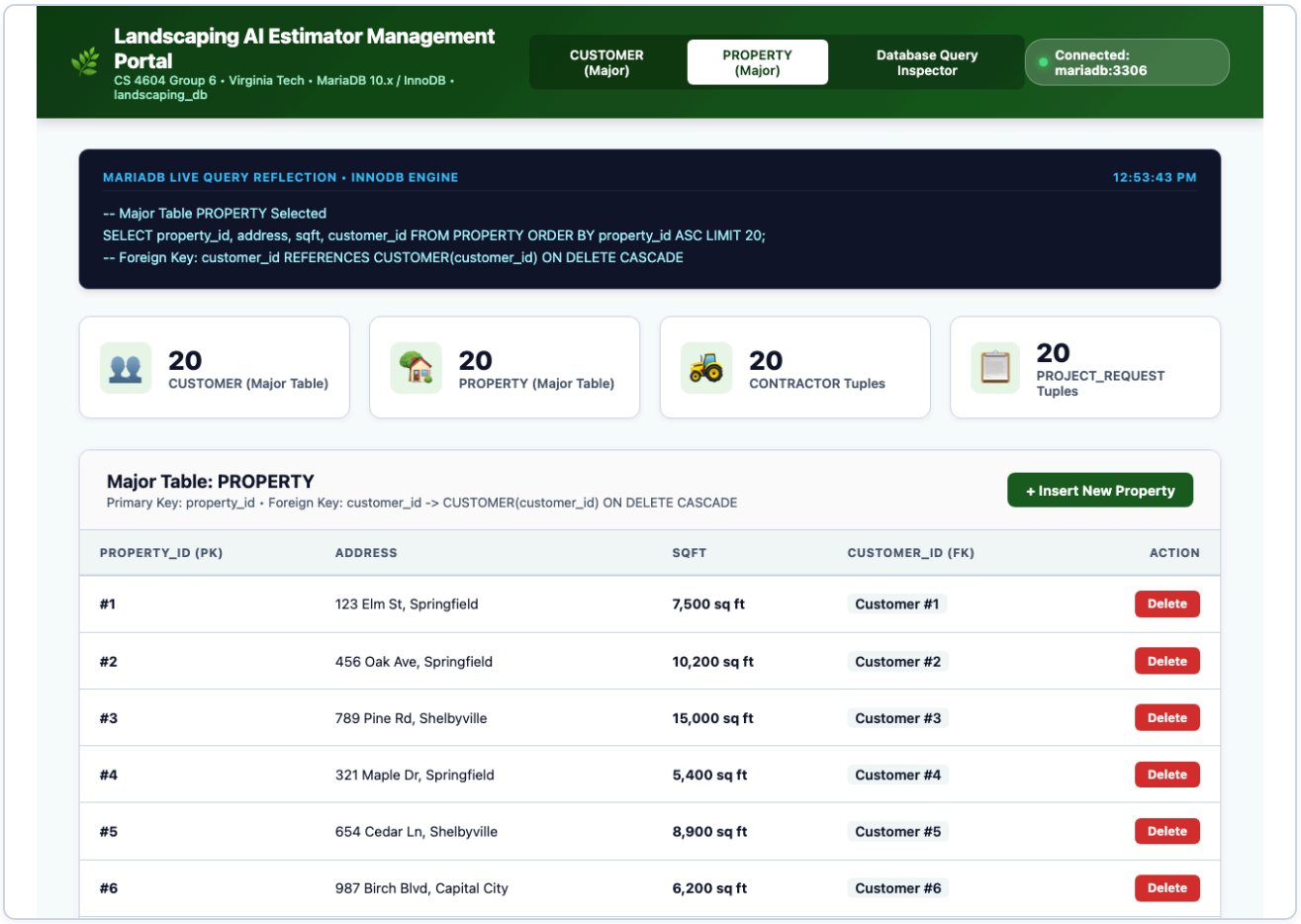


Figure 8: GUI Management view for secondary major table **PROPERTY** with 20 sample tuples, foreign key badges, and Insert/Delete triggers.

7. Deliverables Summary Checklist

Deliverable Requirement	Status	Verification Evidence
0. Connected to Database (Phase 1)	**Verified**	Handshake established with MariaDB `landscaping_db:3306`; status pill and connection log active.
1. Ready Sample Data (20+ Tuples)	**Verified**	20 ready sample tuples loaded in all major tables; documented in Phase 1 and verified in Figure 1 & 8.
2. GUI Insert Functionality	**Verified**	Interactive modal form captures tuple data; auto-increments PK; verified in Figures 2, 3, & 4.

Deliverable Requirement	Status	Verification Evidence
3. GUI Delete Functionality	**Verified**	Row-level delete action with safety confirmation dialog and cascade handling; verified in Figures 5, 6, & 7.
4. Screenshots of GUI & Database Reflection	**Complete**	8 high-resolution screenshots capturing before, during, and after states for both GUI and DB query.
5. Team Member Names & Contributions	**Complete**	Complete breakdown for all 4 team members documented in Section 1.